

INTRODUCTION AU

DÉVELOPPEMENT WEB

FULL-STACK

Approche pratique · React · Tailwind · .NET · SQLite

React

Tailwind CSS

.NET 8

EF Core

SQLite

MVC

DEV

Plan du cours

Ce que nous allons construire ensemble

01 React

Principes, installation, dépendances, structure

02 Tailwind CSS

Installation et configuration rapide

03 Back-end .NET

Architecture MVC, Entity Framework Core

04 SQLite

Base de données légère et embarquée

05 Projet Fil Rouge

Inscription, Connexion, CRUD complet, Tests



React

Bibliothèque JavaScript pour construire des interfaces utilisateur

01

React — Principes fondamentaux

Comprendre le modèle mental avant de coder

Composants

Tout est composant. L'UI est découpée en blocs réutilisables (Header, Button, UserCard...).

Virtual DOM

React maintient un arbre virtuel. Il ne met à jour que les parties réellement modifiées.

JSX

Syntaxe HTML dans du JavaScript. React compile le JSX en appels `React.createElement(...)`.

État (State)

`useState()` gère les données locales. Tout changement de state redéclenche le rendu.

Props

Passage de données parent → enfant.
Immuables dans l'enfant, lecture seule.

Hooks

`useState`, `useEffect`, `useContext`...
Permettent d'utiliser les features React sans classes.

React — Installation avec Vite

npm create vite@latest est la méthode recommandée en 2024

Commandes à exécuter dans le terminal :

```
# 1. Créer le projet React avec Vite
npm create vite@latest mon-app -- --template react

# 2. Entrer dans le dossier
cd mon-app

# 3. Installer toutes les dépendances
npm install

# 4. Lancer le serveur de développement (port 5173)
npm run dev
```

BASH

Pourquoi Vite ?

Ultra-rapide grâce à l'ES Module natif. Hot Module Replacement (HMR) quasi-instantané.

Node.js requis

Version ≥ 18 recommandée. Vérifier : node -v et npm -v avant de commencer.

⚠ Ne pas utiliser Create React App (CRA) — déprécié depuis 2023

React — Comprendre les dépendances

Le fichier package.json est le cœur du projet

```
{
  "name": "mon-app",
  "scripts": {
    "dev": "vite",          // Lance le serveur dev
    "build": "vite build",   // Build de production
    "preview": "vite preview" // Aperçu du build
  },
  "dependencies": {
    "react": "^18.3.0",    // La lib React
    "react-dom": "^18.3.0" // Rendu dans le DOM
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.x", // Plugin Vite
    "vite": "^5.x"                 // Bundler
  }
}
```

JSON

Commandes npm utiles

npm install

Installe toutes les dépendances du package.json

npm install axios

Ajoute axios dans dependencies

**npm install -D
eslint**

Ajoute ESLint dans devDependencies

npm uninstall pkg

Supprime un package

npm update

Met à jour vers les dernières versions autorisées

npm run build

Génère le dossier dist/ prêt pour production

React — Structure du projet

Organisation recommandée pour un projet professionnel

```
mon-app/  
├── public/                # Fichiers statiques servis tel quel  
│   └── favicon.ico  
├── src/  
│   ├── assets/           # Images, fonts, SVGs  
│   ├── components/       # Composants réutilisables  
│   │   ├── Button.jsx  
│   │   └── Navbar.jsx  
│   ├── pages/            # Pages (une par route)  
│   │   ├── Home.jsx  
│   │   ├── Login.jsx  
│   │   └── Register.jsx  
│   ├── services/         # Appels API (axios...)  
│   │   └── authService.js  
│   ├── App.jsx           # Composant racine + Router  
│   └── main.jsx          # Point d'entrée (ReactDOM.render)  
├── index.html            # Template HTML unique  
├── package.json          # Dépendances et scripts  
└── vite.config.js        # Configuration Vite
```

TREE

```
// src/main.jsx — Point d'entrée  
import React from 'react'  
import ReactDOM from 'react-dom/client'  
import App from './App.jsx'  
import './index.css'
```

```
ReactDOM.createRoot(  
  document.getElementById('root')  
)  
.render(  
  <React.StrictMode>  
    <App />  
  </React.StrictMode>  
)
```

JSX

```
// Composant fonctionnel simple  
function UserCard({ name, email }) {  
  return (  
    <div className="card">  
      <h2>{name}</h2>  
      <p>{email}</p>  
    </div>  
  )  
}
```

JSX

Tailwind CSS

Utility-first CSS framework — du CSS sans écrire de CSS

02

Tailwind CSS — Installation

3 étapes pour avoir Tailwind dans un projet Vite + React

```
# Étape 1 – Installer Tailwind et ses outils
npm install -D tailwindcss postcss autoprefixer
```

BASH

```
# Étape 2 – Générer les fichiers de config
npx tailwindcss init -p
# Crée tailwind.config.js et postcss.config.js
```

```
// tailwind.config.js
/** @type {import('tailwindcss').Config} */
export default {
  content: [
    "./index.html",
    "./src/**/*..{js,jsx,ts,tsx}", // Analyse ces
    fichiers
  ],
  theme: {
    extend: {}, // Personnalisations ici
  },
  plugins: [],
}
```

JS

```
/* src/index.css – Remplacer tout le contenu*/
@tailwind base;
@tailwind components;
@tailwind utilities;

/* Tailwind injecte ses styles ici au build */
```

CSS

```
// Exemple d'utilisation dans un composant
function Button({ children }) {
  return (
    <button className="bg-blue-500 hover:bg-blue-700
      text-white font-bold py-2 px-4 rounded">
      {children}
    </button>
  )
}
```

JSX

Tailwind CSS — Classes utilitaires clés

Apprendre en regardant des exemples concrets

Mise en page

flex, grid, block
w-full, h-screen
p-4, m-2, px-6 py-3
gap-4, space-x-2
justify-center, items-center

Typographie

text-sm, text-lg, text-2xl
font-bold, font-medium
text-gray-700, text-blue-500
leading-relaxed
truncate, line-clamp-2

Couleurs & Fond

bg-white, bg-gray-100
bg-blue-500, bg-red-400
border, border-gray-200
rounded, rounded-lg, rounded-full
shadow, shadow-md, shadow-xl

États & Réponse

hover:bg-blue-700
focus:ring-2
focus:outline-none
disabled:opacity-50
sm:w-full md:w-1/2
dark:bg-gray-800

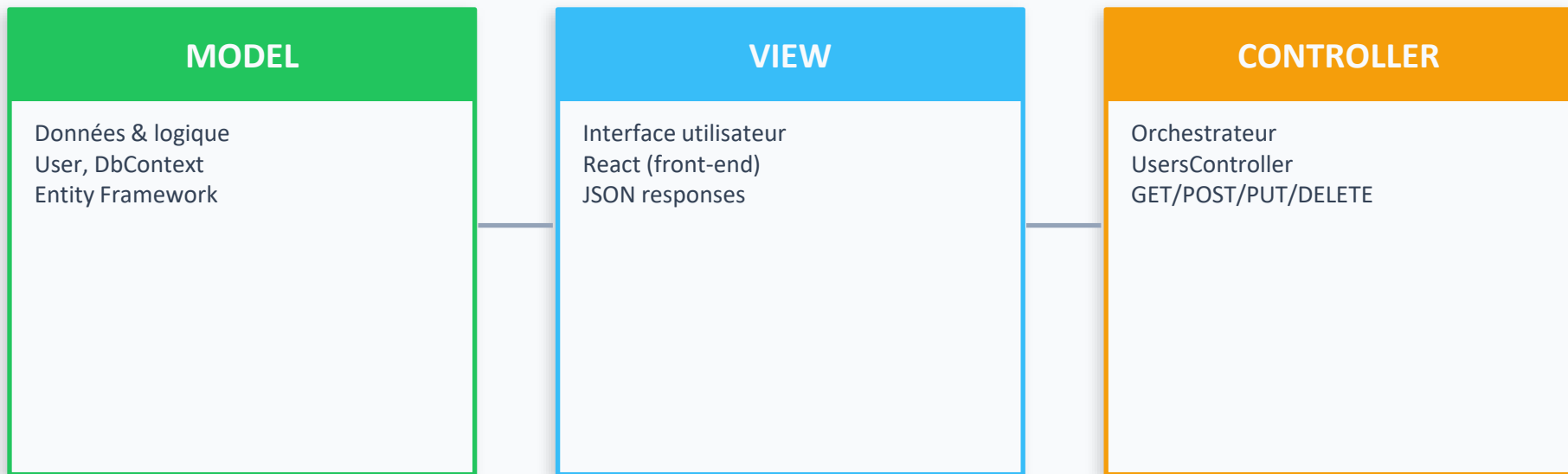
Back-end .NET

API REST avec ASP.NET Core, MVC et Entity Framework Core

03

Architecture MVC

Model · View · Controller — séparation des responsabilités



← Requête HTTP → Controller interroge le Model → Renvoie JSON au View

.NET — Création du projet API

dotnet new webapi — squelette prêt en 30 secondes

```
# Vérifier l'installation de .NET SDK
dotnet --version # >= 8.0
```

BASH

```
# Créer le projet Web API
dotnet new webapi -n MonApp.Api
cd MonApp.Api
```

```
# Installer Entity Framework Core + SQLite
dotnet add package Microsoft.EntityFrameworkCore
dotnet add package Microsoft.EntityFrameworkCore.Sqlite
dotnet add package Microsoft.EntityFrameworkCore.Design
```

```
# Lancer le projet (port 5000 ou 7xxx HTTPS)
dotnet run
```

```
# Générer une migration
dotnet ef migrations add InitialCreate
dotnet ef database update
```

```
// Program.cs — Configuration minimale#
var builder =
    WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddDbContext<AppDbContext>(opt =>
    opt.UseSqlite("Data Source=app.db"));

// CORS pour React (dev)
builder.Services.AddCors(opt =>
    opt.AddPolicy("Dev",
        p => p.AllowAnyOrigin()
            .AllowAnyHeader()
            .AllowAnyMethod()));

var app = builder.Build();
app.UseCors("Dev");
app.MapControllers();
app.Run();
```

Structure du projet .NET :

MonApp.Api/ |— Controllers/ |— Models/ |— Data/ |— Program.cs |— app.db

Entity Framework Core + SQLite

ORM qui traduit vos classes C# en tables SQL

C#

```
// Models/User.cs
public class User
{
    public int Id { get; set; }

    [Required]
    public string FirstName { get; set; } =
string.Empty;
    public string LastName { get; set; } =
string.Empty;

    [Required, EmailAddress]
    public string Email { get; set; } = string.Empty;

    [Required]
    public string Password { get; set; } = string.Empty;
// hashé!

    public string Address { get; set; } = string.Empty;
    public string Phone { get; set; } = string.Empty;
    public string City { get; set; } = string.Empty;
    public string Country { get; set; } = string.Empty;
    public DateTime CreatedAt { get; set; } =
```

```
DateTime.UtcNow;
}
```

C#

```
// Data/AppDbContext.cs
using Microsoft.EntityFrameworkCore;

public class AppDbContext : DbContext
{
    public AppDbContext(
        DbContextOptions<AppDbContext> options)
        : base(options) { }

    // EF Core crée la table "Users"
    public DbSet<User> Users { get; set; }
}
```



Migrations EF Core

EF Core lit vos classes C# et génère le SQL correspondant. La commande `dotnet ef migrations add` crée un fichier C# décrivant les changements. `dotnet ef database update` exécute ce SQL sur SQLite.

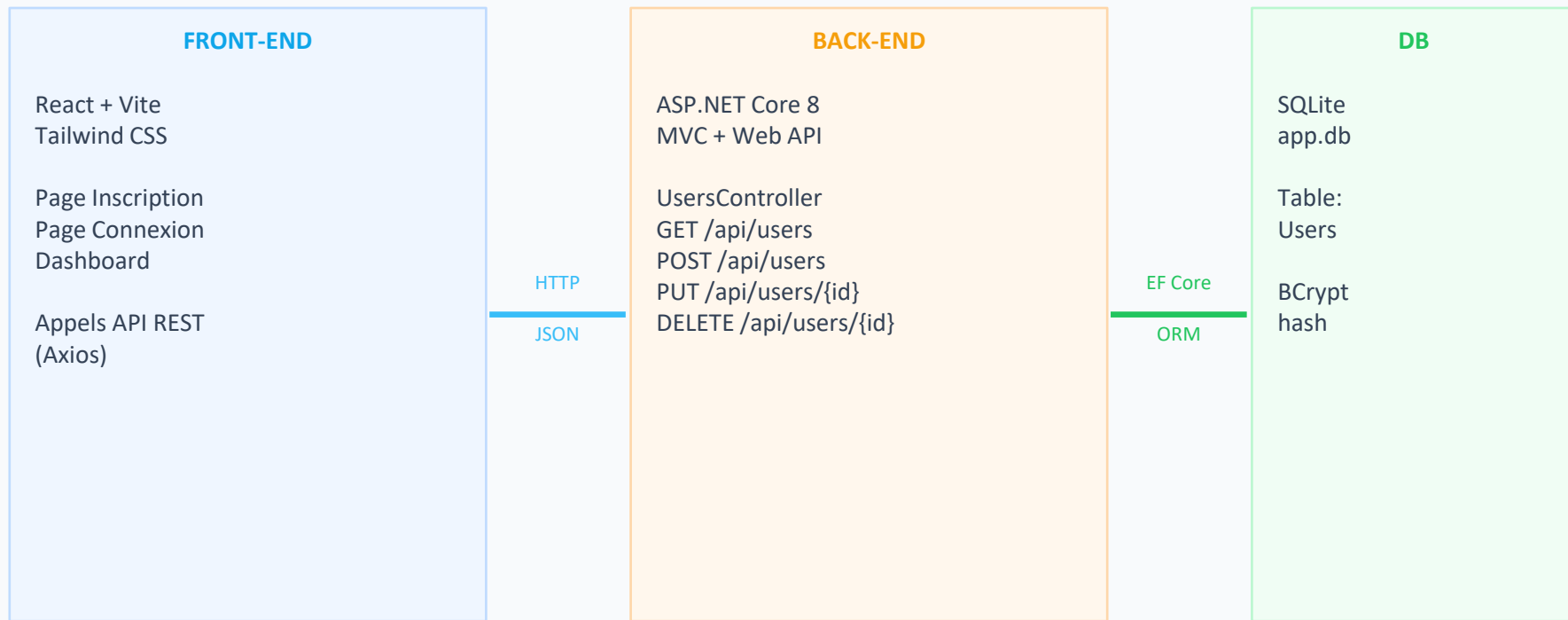
Projet Fil Rouge Exemple

Système d'inscription et de connexion — full-stack complet

04

Vue d'ensemble du projet

Ce que nous allons construire étape par étape



✓ Inscription · ✓ Connexion · ✓ CRUD complet · ✓ Tests unitaires

CRUD — UsersController

Les 4 opérations fondamentales sur les utilisateurs

C#

```
[ApiController]
[Route("api/[controller]")]
public class UsersController : ControllerBase
{
    private readonly AppDbContext _db;
    public UsersController(AppDbContext db) => _db = db;

    // — GET ALL —————
    [HttpGet]
    public async Task<IActionResult> GetAll()
        => Ok(await _db.Users.ToListAsync());

    // — GET BY ID —————
    [HttpGet("{id}")]
    public async Task<IActionResult> GetById(int id)
    {
        var user = await _db.Users.FindAsync(id);
        return user is null ? NotFound() : Ok(user);
    }
}
```

C#

```
// — POST (Inscription)

[HttpPost]
public async Task<IActionResult>
Create(
    [FromBody] CreateUserDto dto)
{
    var user = new User {
        FirstName = dto.FirstName,
        LastName = dto.LastName,
        Email = dto.Email,
        Password =
            BCrypt.Net.BCrypt
                .HashPassword(dto.Password),
        // ... autres champs
    };
    _db.Users.Add(user);
    await _db.SaveChangesAsync();
    return
        CreatedAtAction(nameof(GetById),
```

GET — Récupère

POST — Crée

PUT — Modifie

DELETE — Supprime

CRUD — PUT et DELETE

Modifier et supprimer un utilisateur

```
// — PUT (Modification)
[HttpPut("{id}")]
public async Task<IActionResult> Update(
    int id, [FromBody] UpdateUserDto dto)
{
    var user = await _db.Users.FindAsync(id);
    if (user is null) return NotFound();

    user.FirstName = dto.FirstName ?? user.FirstName;
    user.LastName = dto.LastName ?? user.LastName;
    user.Email = dto.Email ?? user.Email;
    user.Address = dto.Address ?? user.Address;
    user.Phone = dto.Phone ?? user.Phone;
    user.City = dto.City ?? user.City;
    user.Country = dto.Country ?? user.Country;

    await _db.SaveChangesAsync();
    return Ok(user);
}
```

C#

```
// — DELETE
[HttpDelete("{id}")]
public async Task<IActionResult> Delete(int id)
{
    var user = await _db.Users.FindAsync(id);
    if (user is null) return NotFound();

    _db.Users.Remove(user);
    await _db.SaveChangesAsync();
    return NoContent(); // 204
}
```

C#

Codes HTTP courants

200 OK	Succès général (GET, PUT)
201 Created	Ressources créée (POST)
204 No Content	Supprimé avec succès
400 Bad Request	Données invalides
404 Not Found	Ressource introuvable

💡 DTO (Data Transfer Object) : classe légère qui définit uniquement les champs acceptés par l'API. Évite d'exposer le modèle entier.

Authentification — Hash du mot de passe

Ne jamais stocker un mot de passe en clair — BCrypt

```
# Installer BCrypt
dotnet add package BCrypt.Net-Next
```

BASH

```
// Inscription – Hash à la création
string motDePasseHash = BCrypt.Net.BCrypt
    .HashPassword(dto.Password);
// ex. résultat: "$2a$11$AbCdEfGhIjKlMnOpQrStUu..."
user.Password = motDePasseHash;

// -----
// Connexion – Vérification [HttpPost("login")]
[HttpPost("login")]
public async Task<IActionResult> Login(
    [FromBody] LoginDto dto)
{
    var user = await _db.Users
        .FirstOrDefaultAsync(u => u.Email == dto.Email);

    if (user is null)
        return Unauthorized("Email introuvable");

    bool valid = BCrypt.Net.BCrypt
        .Verify(dto.Password, user.Password);
```

C#

Comment fonctionne BCrypt ?

1 Salt aléatoire

BCrypt génère un salt unique par mot de passe

2 Hash + Salt

Combine le mot de passe + salt, puis applique Blowfish

3 Coût (cost factor)

2^{11} = 2048 itérations. Rend les attaques lentes

4 Résultat unique

Même mot de passe → hash différent à chaque fois

5 Verify()

Extrait le **salt** du hash stocké et recalcul

● Ne jamais retourner le champ Password dans les réponses API !

Front-end React — Formulaire d'inscription

Appel API POST avec Axios et gestion d'état

```
// src/pages/Register.jsx
import { useState } from 'react'
import axios from 'axios'

export default function Register() {
  const [form, setForm] = useState({
    firstName:'', lastName:'', email:'',
    password:'', address:'', phone:'',
    city:'', country:''
  })
  const [error, setError] = useState('')
  const [success, setSuccess] = useState(false)

  const handleChange = e =>
    setForm({ ...form, [e.target.name]: e.target.value })

  const handleSubmit = async e => {
    e.preventDefault()
    try {
      await axios.post('/api/users', form)
      setSuccess(true)
    } catch (err) {
      setError(err.response?.data || 'Erreur serveur')
    }
  }
}
```

JSX

```
# Installer Axios
npm install axios
```

Axios = bibliothèque HTTP pour les appels API depuis React.

```
return (
  <form onSubmit={handleSubmit}
    className="max-w-md mx-auto p-6
      bg-white rounded-xl shadow">
    <h1 className="text-2xl font-bold mb-6">
      Inscription
    </h1>

    {error && (
      <div className="text-red-500 mb-
4">{error}</div>
    )}

    <input name="firstName" type="text"
      placeholder="Prénom"
      className="w-full border p-2 rounded mb-
3"
      onChange={handleChange} required />

    { /* ... autres champs ... */ }

    <button type="submit"
      className="w-full bg-blue-500 text-white
        p-2 rounded hover:bg-blue-
700">
      S'inscrire
    </button>
  </form>
)
}
```

JSX

Front-end React — Formulaire de connexion

Appel API POST /login et stockage de la session

```
// src/pages/Login.jsx
import { useState } from 'react'
import { useNavigate } from 'react-router-dom'
import axios from 'axios'

export default function Login() {
  const [email, setEmail] = useState('')
  const [password, setPassword] = useState('')
  const [error, setError] = useState('')
  const navigate = useNavigate()

  const handleSubmit = async e => {
    e.preventDefault()
    try {
      const res = await axios.post('/api/users/login',
        { email, password })

      // Stocker l'userId en session
      sessionStorage.setItem('userId', res.data.userId)
      navigate('/dashboard') // Redirection

    } catch (err) {
      setError('Email ou mot de passe incorrect')
    }
  }
}
```

JSX

```
return (
  <div className="min-h-screen flex items-center
    justify-center bg-gray-100">
    <form onSubmit={handleSubmit}
      className="bg-white p-8 rounded-xl
        shadow-md w-full max-w-sm">
      <h1 className="text-2xl font-bold mb-6
        text-center">Connexion</h1>
      {error &&
        <p className="text-red-500 mb-4">{error}</p>}
      <input type="email" placeholder="Email"
        value={email}
        onChange={e => setEmail(e.target.value)}
        className="w-full border p-2 rounded mb-3"
        required />
      <input type="password" placeholder="Mot de
        passe"
        value={password}
        onChange={e => setPassword(e.target.value)}
        className="w-full border p-2 rounded mb-4"
        required />
      <button type="submit"
        className="w-full bg-indigo-600 text-white
          p-2 rounded hover:bg-indigo-700">
        Se connecter
    </form>
  </div>
)
```

En production : utiliser JWT (JSON Web Token) à la place de sessionStorage pour une sécurité accrue.

Tests Unitaires

Valider le comportement de l'API — xUnit + Moq

05

Tests Unitaires .NET — xUnit

Tester chaque fonction de manière isolée

```
# Créer le projet de tests
dotnet new xunit -n MonApp.Tests
cd MonApp.Tests
```

BASH

```
# Référencer le projet API
dotnet add reference ../MonApp.Api/MonApp.Api.csproj
```

```
# Installer Moq (mock des dépendances)
dotnet add package Moq
dotnet add package
```

```
// Tests/UsersControllerTests.cs
public class UsersControllerTests
{
    private AppDbContext GetInMemoryDb()
    {
        var opts = new
DbContextOptionsBuilder<AppDbContext>()
            .UseInMemoryDatabase(
                databaseName: Guid.NewGuid().ToString())
            .Options;
        return new AppDbContext(opts);
    }
}
```

C#

```
# Lancer tous les tests
dotnet test
```

```
[Fact]
public async Task Create_HashesPassword()
{
    using var db = GetInMemoryDb();
    var ctrl = new UsersController(db);
    var dto = new CreateUserDto {
        FirstName = "Bob", LastName = "Martin",
        Email      = "bob@test.com",
        Password   = "monMotDePasse123"
    };

    await ctrl.Create(dto);

    var user = await db.Users.FirstAsync();
    // Le mot de passe NE DOIT PAS être stocké tel
    quel
    Assert.NotEqual("monMotDePasse123",
user.Password);
    // Doit être un hash BCrypt valide
    Assert.True(BCrypt.Net.BCrypt
        .Verify("monMotDePasse123", user.Password));
}
```

C#

RÉCAPITULATIF — Ce que vous savez faire maintenant



React + Vite

Créer un projet, gérer les dépendances

Composants fonctionnels, JSX, hooks

Formulaires contrôlés + appels API Axios



Tailwind CSS

Installer et configurer Tailwind

Classes utilitaires flex/grid/spacing/colors

Responsive design avec sm: md: lg:



ASP.NET Core

Architecture MVC, WebAPI

Entity Framework Core + SQLite

Migrations et DbContext



Auth & CRUD

Hash BCrypt sécurisé

GET / POST / PUT / DELETE

Codes HTTP, DTOs, réponses typées



Tests xUnit

Base de données InMemory

AAA : Arrange, Act, Assert

Valider hash, 404, collections